Marc, Daniel and Rachel—will be assigned to a row of three adjacent lockers." Followed by multiple conditions that govern the assignment: e.g. "Each locker must be assigned to either one or two children. Fred must share a locker, and each shared locker must be assigned to one girl and one boy." etc. Questions ask about properties of the solution, for instance "Which one of the following is a complete and accurate list of the children who must be among those assigned to shared lockers??" with "Fred, Rachel" as the corresponding answer. In general, for this scenario, inputs will describe such constraint satisfaction problems followed by multi-answer questions. Accuracy for this scenario is measured using exact matching.

Data. Zhong et al. (2021) created the dataset by collecting questions from the analytical reasoning section of prior tests from 1991 to 2016. The train-dev-test splits for the dataset are 1630-231-230 with inputs on average being 168 tokens tokens based on the GPT-2 tokenizer and outputs being a single answer.

Pre-processing. No special pre-processing is performed beyond tokenization.

Data Resources. We access the dataset at `https://github.com/zhongwanjun/AR-LSAT`. The dataset is made available through our benchmark. Other resources on the dataset include (Wang et al., 2022b) and (Wang et al., 2022b).

E.3.11 Entity Matching

Scenario Description. **EntityMatching** (Konda et al., 2016; Li et al., 2020) is a reasoning scenario that features reasoning over if two structured entities from two separate relations (i.e. tables) are the same or not. The is a common preprocessing step in data cleaning pipelines where relations are integrated across various sources and often contain near-duplicate records of the same entity(Golshan et al., 2017). Consider relation A and relation B each with attributes "name" and "age". An example input for **EntityMatching** would be "Relation A is Name: Foo. Age: 37. Relation B is Name: Ffo. Age: 30s. Are relation A and B the same?". The desired output is "Yes". While this scenario is over structured data, we apply serialization techniques similar to (Narayan et al., 2022) and (Li et al., 2020) to turn the input into natural language. In general, matches will be two relations with minor textual differences (e.g., misspellings, nicknames, or format differences). Non-matches will contain more meaningful textual differences.

Accuracy for this scenario is typically measured with F1 using 1/0 outputs for yes/no. However, we use exact match accuracy to handle if the model fails to output yes/no.

Data. We use three of the standard Magellan (Konda et al., 2016) datasets—Beer, Abt Buy, and iTunes-Amazon (dirty)—and average the results across the three datasets. The data comes pre-blocked, meaning match candidates are already generated. The train-dev-test splits of Beer, Abt Buy, and iTunes-Amazon are 269-92-92, 5744-1917-1917, and 322-110-110. The average input token size is 515-838-1255 and output token size is 2.

Pre-processing. For pre-processing, we serialized each row similar to Narayan et al. (2022) and Li et al. (2020) with each attribute serialized by the attribute name, a colon, the value, with an ending period.

Data Resources. We access the dataset at `https://github.com/anhaidgroup/deepmatcher/blob/master/Datasets.md`.

E.3.12 Data Imputation

Scenario Description. **DataImputation** (Mei et al., 2021; Abedjan et al., 2016) is a reasoning scenario that features reasoning over what a missing or NAN value is in a structure row from a relation (i.e. table). The is a common preprocessing step in data cleaning pipelines where relations can contain erroneous and missing values. Consider relation A with attributes "name", "phone", and "city" where the "city" attribute is often unknown. An example input for **DataImputation** would be "Name: Foo. Phone: 503-214-3456. City?" The desired output is "Portland". While this scenario is over structured data, we apply serialization techniques similar to Narayan et al. (2022) to turn the input into natural language. In general, missing

values can be determined through other information in the row. For example, the model can rely on the fact that there is a functional dependency from area code to city to learn the missing city.

Accuracy for this scenario is measured by quasi exact match accuracy.

Data. We use two imputation datasets (Mei et al., 2021)—Buy and Restaurants—and average the results across the three datasets. The train-dev-test splits of Buy and Restaurants are 470-118-66 and 623-157-87. The average input token size is 270-230 and average output token size is 2-3 for each dataset.

Pre-processing. For pre-processing, we follow Mei et al. (2021) and select 10% of rows from the data to be our test set with the city attribute to be imputed. Each attribute is serialized by the attribute name, a colon, the value, with an ending period. We end the input with the column to impute followed by a question mark.

Data Resources. We access the Buy dataset at `https://dbs.uni-leipzig.de/file/Abt-Buy.zip` and the Restaurant dataset at `https://www.cs.utexas.edu/users/ml/riddle/data/restaurant.tar.gz`.

E.4 Memorization & copyright

Scenario Description. This scenario evaluates and assesses the degree to which foundation models regurgitate copyrighted/licensed content in their pretraining data. In particular, we focus on large language models to make evaluation and analysis tractable. We briefly discussed the issue of copyright and the fair use doctrine in the main text. Here, we describe the technical metrics we use for measuring the memorization abilities. While it is not always the case that memorization of copyrighted content implies legal risk, intuitively, the likelihood of risk for a model that has memorized large amounts of content should be higher than one which does not appear to memorize (for use cases not covered by fair use). On a high-level, our evaluation samples a prefix of some copyrighted content which likely appears in the pretraining corpus, samples completions from target models, and measures the overlap between the completion and reference.

Data. We consider three sources for running these "extraction attacks":

(1) Books from the books corpus[106],

(2) popular books from a bestseller list,[107] and

(3) source code of the linux kernel (licensed under GPL).[108]

Pre-processing. We sampled 1k instances for (1), the prompts of which begin from randomly selected sentences. We sampled 20 instances for (2) with all prompts starting with the first paragraph of each book. We sampled 1k instances of (3), the prompt of which begin from randomly selected lines.

Data Resources. We list the sources of the our preprocessed data splits (including prompts and ground truth completion):

- books
- popular books
- source code

[106] `https://github.com/soskek/bookcorpus`
[107] `https://www.theguardian.com/news/datablog/2012/aug/09/best-selling-books-all-time-fifty-shades-grey-compare`
[108] `https://github.com/torvalds/linux`

E.5 Disinformation

Scenario Description. Disinformation is a pair of scenarios based on the work of Buchanan et al. (2021) in which models are prompted to generate text that furthers the goals of disinformation actors. The two disinformation scenarios are narrative reiteration and narrative wedging. Narrative Reiteration tests the ability of models to generate new headlines that promote a given narrative. The prompt consists of a high level thesis statement and five headlines that support that statement. An example thesis statement would be "Medical professionals are exaggerating the threat of COVID.", and a headline that supports this would be "Deaths caused by pneumonia, strokes and heart attacks are being registered as COVID-19 deaths." Models are manually evaluated on their ability to produce additional headlines that support the thesis statement.

Narrative wedging tests the ability of models to generate short, divisive messages that are targeted at particular groups. Specifically, the prompts encourage certain voting behavior among religious groups as well as promote or downplay anti-Black racism. An example prompt would include: "Five reasons why Muslims need to vote Democrat this election" as well as a list of four reasons. For a model to be useful for generating disinformation, its generations must be diverse and of high quality. Diversity is measured using self-BLEU (Zhu et al., 2018) and a Monte-Carlo estimate of entropy. Quality is measured with human evaluation. For reiteration, annotators are asked to evaluate the extent to which the generation supports the thesis and that generation sounds like a headline. For wedging, annotator asked to rate whether the generation is addressed to the correct audience, whether it supports the intended goal, if its style matches that of an internet forum, and if the message is divisive. For full annotation guidelines see `https://worksheets.codalab.org/worksheets/0x5f534427e2ae40c4af683b2457676a39`. Each generation is evaluated by five annotators.

Data. The reiteration dataset is created using headlines from the **MisinfoReactionFrames** dataset (Gabriel et al., 2022). We use false headlines that fall into two topics: climate change, which come from NELA-GT-2018-2020 (Gruppi et al., 2022; Nørregaard et al., 2019) and ClimateFEVER (Bulian et al., 2020); as well as COVID, which come from The CoronaVirusFacts/DatosCoronaVirus Alliance Database [109], and the ESOC Covid19 Misinformation Dataset [110]. 114 headlines about COVID and 147 headlines about climate change from the dataset are manually clustered as pertaining to 38 theses and 49 groups respectively. A thesis are manually written for each group, such that all headlines in the group support the thesis. The reiteration dataset is identical to the hand-specified one used by (Buchanan et al., 2021). There are two types of prompts: the first encourages certain voting behavior (vote Democratic, Republican, or not at all) and are targeted toward religious groups (Christians, Muslims, and Jews). The second encourages division by targeting Black people and anti-Black racists. For reiteration, we use two training instances, and the prompts have 344.581 tokens on for the COVID headlines and 279.135 and for the climate change headlines on average. For wedging we use no training instances, and the prompts have 123.696 tokens on average. (All tokens counts are based on the GPT-2 tokenizer.)

Pre-processing. For reiteration, we preprocess the dataset by randomly selecting two instances to be training instances for each of the COVID and climate headlines. There is no preprocessing for the wedging scenario.

Data Resources. The reiteration dataset is available here, and the wedging dataset is available here. The human annotation results can be viewed at disinformation_reiteration_human_eval.json and disinformation_wedging_human_eval.json,

E.6 Bias

E.6.1 BBQ

BBQ (Bias Benchmark for Question answering; Parrish et al., 2022) is a QA scenario that highlights social biases against specific social groups. As an example, an input for the scenarios looks like: "At the concert

[109] `https://www.poynter.org/`
[110] `https://esoc.princeton.edu/publications/esoc-covid-19-misinformation-dataset`